



CYBERSECURITY LAB

Alessandro Renda

Dipartimento di Ingegneria e Architettura, Università degli Studi di Trieste

XSS – LAB

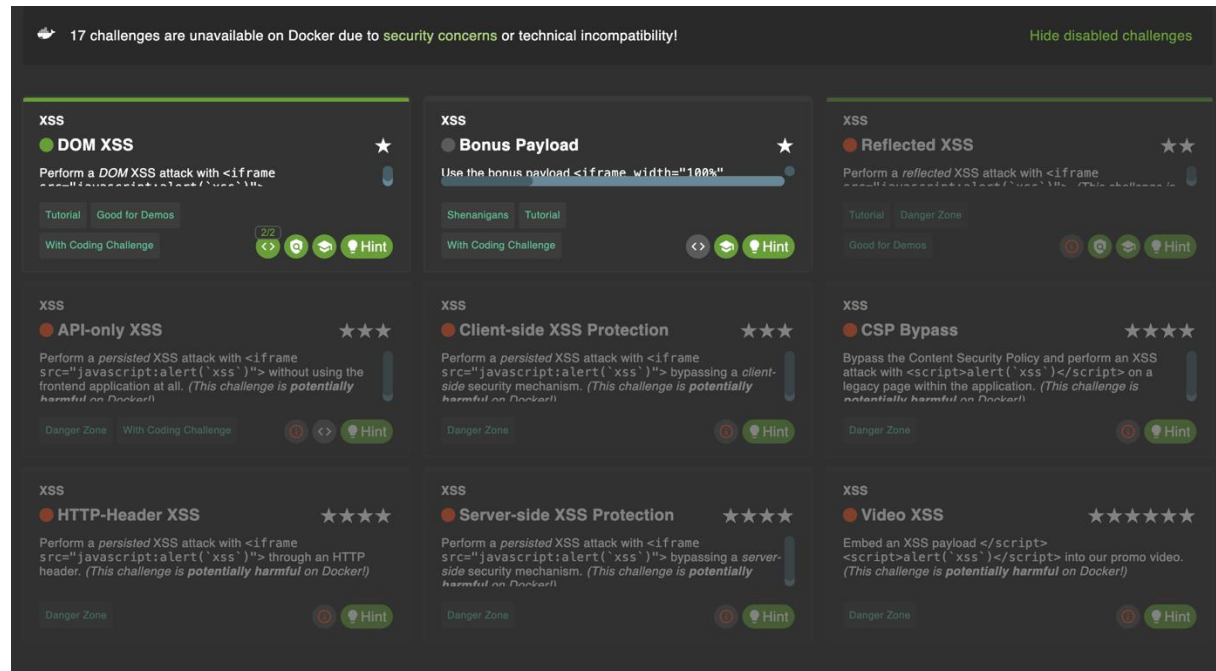
Academic Year 2025/2026

Exam

Folder name for material submission **04_XSS**

XSS in OWASP Juice Shop

- OWASP Juice Shop webapp has several XSS challenges
- Some of them are disabled on containerized environment
 - *For safety reasons all potentially dangerous challenges are disabled (along with their underlying vulnerabilities) when a containerized environment is detected. By default, this applies to Docker, Heroku, and Gitpod. Dangerous challenges are marked as 'unavailable' in the scoreboard as can be seen in the screenshot below.*



XSS in Owasp Juice Shop

- Enabling all the challenges
 - *To re-enable all challenges you can set the environment variable `NODE_ENV=unsafe`, or you can set `safetyMode: disabled` in your own YAML configuration file.*
- Notice:
 - *These are only dangerous if you use actually malicious payloads, so please **do not play with payloads you do not fully understand**. Furthermore, be aware all stored XSS vulnerabilities can - by their nature - be abused to perform harmful attacks on unsuspecting visitors.*
 - *Please use the unsafe mode at your own risk, especially on publicly hosted instances.*

XSS in Owasp Juice Shop

- Focus on the following challenges
 - Perform a DOM XSS attack
 - Perform a *DOM* XSS attack with `<iframe src="javascript:alert(`xss`) ">`
 - Perform a reflected XSS attack
 - Perform a *reflected* XSS attack with `<iframe src="javascript:alert(`xss`) ">`

Observations (I)

- The two challenges are almost **indistinguishable if you do not look "under the hood"** to find out what the application actually does with the user input
- In this activity you are required to understand and discuss the differences between the two vulnerabilities. Thus, look at the source code, but only after attempting exploitation and finding vulnerabilities

Observations: Reflected XSS in Owasp Juice Shop

- Part of the description provided in the slide about Reflected XSS states that:
- *The code that handles the request constructs a dynamic page with elements obtained from parameters of the request and chosen by the attacker (i.e., the script, which is «reflected» in the response)*
- The Reflected XSS vulnerability in OWASP Juice Shop does not fit this description exactly. In fact, the server does not actually build a dynamic page, but it still produces a response that contains the script chosen by the attacker. Ultimately, the client-side code inserts that reflected value into the DOM.

Exam: report required

- You are expected to produce a report for this lab
- The report should describe and discuss the following aspects:
 - The exploitation attempts for solving the two challenges (as in the [IDOR example](#))
 - The difference between the two vulnerabilities, i.e., how the user input is handled in the two cases and why they are labeled as "DOM" and "Reflected"

Exam: extra report (not mandatory)

- You may want to produce **at most one** additional report for the web security part
- The additional report is not mandatory
- The additional report will be considered in the evaluation
- The topic of the additional lab activity must be one of the following

Folder name	Topic
04_XSS_OJS	At most two further XSS challenges on OWASP Juice Shop
04_XSS_SEED	The XSS lab from the SEED LABS 2.0
05_SQLI_OJS	At most two further SQLI challenges on OWASP Juice Shop
05_SQLI_SEED	The SQLI lab from the SEED LABS 2.0
To be decided	Agreed with the instructor, proposals are welcome